

Encapsulation

Programming and Data Structures — Week 1, Lecture 2

Dr. Innocent Nyalala

Last time, and the gap it leaves

```
p = Point(3, 4)
p.x = "banana"          # legal Python. Nothing stops this.
p.distance_to(Point(0, 0)) # crashes, far from where the mistake was made
```

Point from Lecture 1 has attributes anyone can read *and rewrite* freely. Nothing about the Point class prevents x from becoming a string. The bug surfaces later, in a different method, which makes it harder to find than it should be.

Encapsulation: hide the data, expose the behavior

```
1 class BankAccount:
2     def __init__(self, balance=0):
3         self._balance = balance      # leading underscore: "do not touch directly"
4
5     def deposit(self, amount):
6         if amount <= 0:
7             raise ValueError("deposit must be positive")
8         self._balance += amount
9
10    def withdraw(self, amount):
11        if amount > self._balance:
12            raise ValueError("insufficient funds")
13        self._balance -= amount
14
15    def balance(self):
16        return self._balance
```

Encapsulation means an object's internal state is only ever changed through its own methods — never poked at directly from outside.

Why the leading underscore

```
account = BankAccount(100)
account.deposit(50)
print(account.balance())      # 150 - the only sanctioned way to read it
```

Python has no private keyword. `_balance` is a **convention**: it tells every other programmer “this is internal, go through the methods instead.” Python will not stop you from writing `account._balance = -500`, but a colleague who does that is violating a documented agreement, not exploiting a loophole you forgot to close.

Tracing a bad withdrawal

```
account = BankAccount(100)
account.withdraw(1000)
```

Walk through what happens: `withdraw` runs, checks `1000 > self._balance` — that is `1000 > 100`, which is `True` — and raises `ValueError` **before** the line that would have subtracted. The balance never becomes `-900`. This is the guarantee encapsulation buys you: invalid states become impossible to reach, not just unlikely.

A second worked example: enforcing a valid range

```
class Temperature:
    """Stores a temperature in Celsius, refusing physically impossible values."""

    ABSOLUTE_ZERO_C = -273.15

    def __init__(self, celsius):
        if celsius < self.ABSOLUTE_ZERO_C:
            raise ValueError(f"{celsius} C is below absolute zero")
        self._celsius = celsius
```

```
def celsius(self):  
    return self._celsius  
  
def fahrenheit(self):  
    return self._celsius * 9 / 5 + 32
```

Temperature(-300) raises immediately. Without this class, -300 sits in a plain variable until it reaches whatever calculation eventually breaks.

What encapsulation is not

It is **not** about secrecy for its own sake, and it is not the same as security. Anyone reading the source code can see `_balance`. The point is a **social and structural contract**: the only correctness guarantees the class can make are the ones enforced in its own methods, so route every change through them.

Exercise (do this before the lab)

Add a `set_celsius(self, new_value)` method to `Temperature` that re-checks the absolute-zero constraint before accepting the new value. Then explain in one sentence why allowing direct assignment to `self._celsius` from outside the class would make this method pointless.

Before next lecture

`BankAccount` and `Temperature` are unrelated classes that happen to share a shape: hide state, expose checked operations. Next lecture: what happens when several classes are not just similarly shaped but are genuinely *related* — a `Circle` and a `Square` are both, unmistakably, shapes.